Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

CPU Scheduling

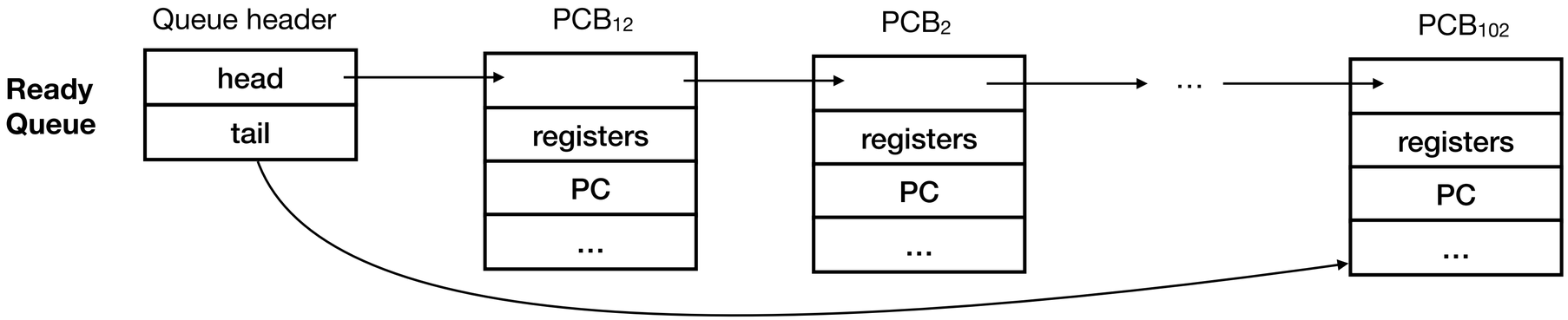
University of Minho

2025-2026

Previously, in your OS class

OS Scheduler

- The number of processes ready to run is significantly higher than the number of available CPU cores
- The PCBs of *Ready* processes are kept in a **Ready queue**
- The **OS scheduler** is responsible for choosing the next process to execute
  - But what about the **policies**? What process should be scheduled next?



Previously, in your OS class

Some algorithms

- **FCFS (FIFO)** - Schedule the process that arrived first (earlier)
  - Simple and easy to implement!
  - Turnaround time is sensitive to the order of arrival
- **SJF** and **STCF** - Schedule the process with the shortest (remaining) runtime
  - Improve turnaround time. STCF is better when processes arrive at distinct times
  - Both require knowing the runtime in advance, and are bad for response time
- **RR** - Run a process for a given time slice (*quantum*), then switch to the next
  - A fair scheduler, good for improving response time (good for interactive systems)
  - Bad algorithm for turnaround time (fair algorithms are bad for this metric)
- **MLFQ** - Dynamically adjusts the priority of processes based on their behavior
  - Prioritizes short-duration processes, without knowing their runtime in advance
  - Several tuning knobs (*e.g.*, number of queues, allotment, priority boost, …)

CPU Scheduling

Proportional-share (aka fair-share) schedulers

- **Goal:** give each job a fair percentage of CPU (**fairness** metric)
- **Challenges**
  - How can we design a scheduler to share the CPU in a proportional manner?
  - What are the key mechanisms for doing so?
  - How effective are they?
- Interesting algorithms
  - Lottery Scheduling
  - Stride Scheduling
  - Completely Fair Scheduler (CFS)
  - Earliest Eligible Virtual Deadline First (EEVDF)

Lottery Scheduling

Overview

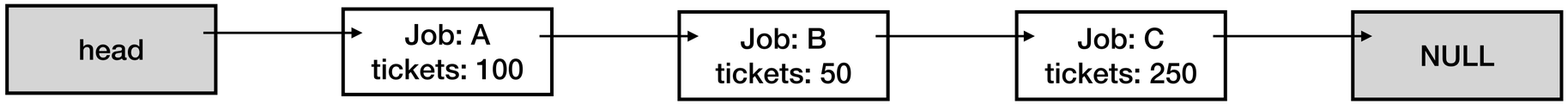
- **Ticket:** the share of a resource (e.g., CPU, memory, disk) that a process (or user) should receive
  - The percent of tickets that a process has represents its share of the system resource
- Example: *Process A* gets 75% of CPU time and *Process B* gets 25%
  - The system has 100 tickets (0-99). *Process A* has tickets 0-74 and *Process* 75-99
  - When the established CPU time slice ends, a lottery is held
    i.e., the scheduler picks a random number between 0 and 99
- Probabilistic algorithm, using randomness
  - Fast algorithm that requires little state!
  - Randomness ensures probabilistic correctness in achieving the desired proportion…
    … but it may require several iterations to converge to the desired percentages

Winning tickets	*63*	*85*	*70*	*39*	*76*	*17*	*29*	*41*	*36*	*39*	*10*	*99*	*68*	*83*	*63*	*62*	*43*	*0*	*49*	*12*
Schedule	*A*	*B*	*A*	*A*	*B*	*A*	*A*	*A*	*A*	*A*	*A*	*B*	*A*	*B*	*A*	*A*	*A*	*A*	*A*	*A*

Lottery Scheduling

Implementation(s)

- *Example: Processes A*, *B,* and *C* have 100, 50, and 250 tickets, respectively
  - A **random generator** generates a number between 0 and 400 (total number of tickets). Let's assume 300 is generated
  - A **counter** is kept while iterating through the **data structure** (e.g., a list)
    1. As 100 < 300, the counter is updated to 100 after passing through Job A
    2. As 150 (100+50) < 300, the counter is updated to 150 after passing through Job B
    3. As 400 > 300 (counter's value exceeds the winning ticket's value), then Job C is scheduled
- Questions:
  - If a list is being used, would it help to order it from the highest number of tickets to the lowest?
  - What if an array structure were used instead? What would be the advantages and drawbacks?



Stride Scheduling

Overview

- What when randomness does not deliver the exact percentages?
- **Stride Scheduling** does not require randomness
  - Each job in the system has a **stride** value, inverse in proportion to the number of tickets it has
  - Every time a job runs, its **pass** value (a counter) is incremented by its stride value
  - **Scheduling policy:** Schedule the job with the lowest pass value
- *Example: Processes A, B,* and *C* have 100, 50, and 250 tickets
  - The stride is computed by dividing some large number (e.g., 10 000) by the number of tickets each process has been assigned
  - Stride values for A, B, and C: 100, 200, and 40
  - A random process is chosen when the pass values are identical

Pass (A) stride=100	Pass (B) stride=200	Pass (C) stride=40	Who runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	…

Example

Comparison

Stride vs lottery scheduling

- Lottery scheduling ensures **probabilistic correctness**, while stride scheduling is **deterministic**
- Stride scheduling requires maintaining **more global state**
  - What if a new process enters in the middle of our example?
    - Should the pass value be set to 0?
  - With the lottery scheduling, it is easier to incorporate new processes
    - Add them to the data structure, update the total amount of tickets, and that's it

Pass (A) stride=100	Pass (B) stride=200	Pass (C) stride=40	Who runs?
0	0	0	A
100	0	0	B
100	200	0	C
100	200	40	C
100	200	80	C
100	200	120	A
200	200	120	C
200	200	160	C
200	200	200	...

Example

Completely Fair Scheduler (CFS)

Overview

- **CFS** is one of the most used fair schedulers
  - Used by default in several Linux distributions (up until Linux 6.6)
- Goals
  - Fairly divide a CPU evenly among all competing processes
  - Be very efficient and scalable in choosing the next process to run

Completely Fair Scheduler (CFS)

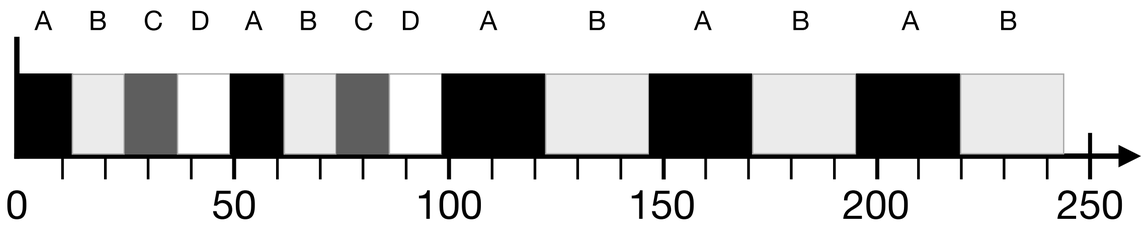
Virtual runtime

- **Virtual runtime (*vruntime*):** counts the CPU time used by a given process
  - Incremented every time a process uses the CPU
  - Increments do not need to follow real CPU runtime!
- **Selection criteria:** choose the process with the lowest vruntime
- *vruntime* may be incremented differently for specific processes
  - **Example:** time increases more slowly for high-priority processes
  - Priority may be chosen by users and system administrators (**nice** level of a process)
    - The nice level can be set from -20 to +19, with a default of 0.
    - Positive nice values imply lower priority, and negative values imply higher priority
  - CFS maps the nice value of each process to a **weight** parameter
    - The *weight* is included in the *vruntime* calculation and on the time slice attributed to each process

Completely Fair Scheduler (CFS)

Time-slicing

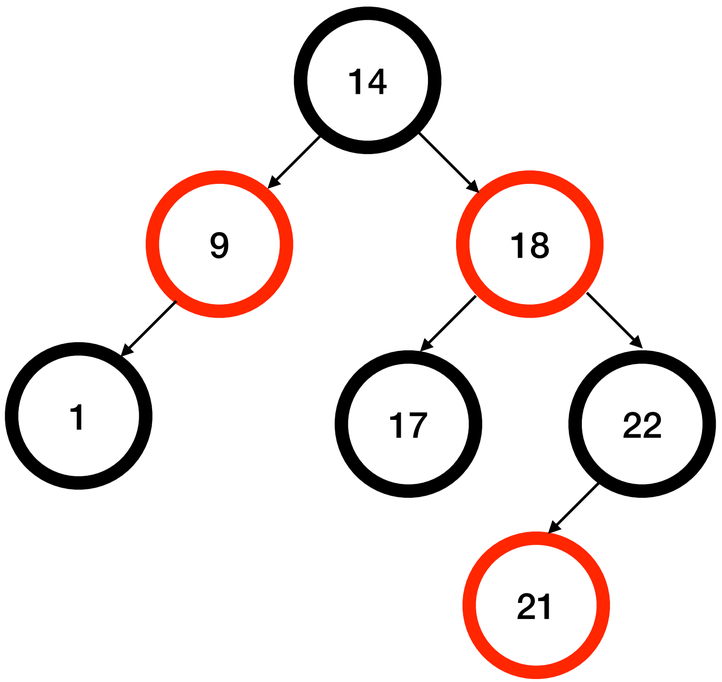
- **Scheduling latency target (sched_latency):** Within the *sched_latency* time interval, each process should have a fair chance to use the CPU
  - **Example:** Assuming 4 processes and *sched_latency = 48 ms*, each process is given a time slice of 12 ms
- The time-slice adjusts dynamically when processes arrive/leave the system
  - **Example:** if 2 processes finish executing, the time slice for each of the two remaining processes is increased to 24 ms
- A minimum time slice is set when a large number of processes are waiting (**min_granularity**)
  - **Question:** Do you know why?
- A periodic timer interrupt (e.g., every 1 ms) allows CFS to wake up and check if the current job has reached its time limit



Completely Fair Scheduler (CFS)

Red-black tree

- Searching through a list for the process with the lowest *vruntime* does not scale
- A **red-black tree** is used to index processes by their *vruntime* and to efficiently choose the next to run
  - A balanced tree that maintains low depths, ensuring search operations are logarithmic in time
  - Only ready processes are kept in the tree
- If a process goes to sleep for a very long time, when it returns, its vruntime may be significantly smaller than that of other running processes
  - It can monopolize the CPU for a long time!
  - CFS adjusts the *vruntime* of a waking process to the minimum value found in the tree!



CFS Red-Black tree

Earliest Eligible Virtual Deadline First (EEVDF)

Overview

- Linux default scheduler (since Linux 6.6)
- While being a fair scheduler, it aims to provide better responsiveness for interactive workloads than CFS
  - CFS picks the **smallest vruntime**
  - EEVDF adds an eligibility scheduling filter (**lag**) and picks the **earliest virtual deadline** among running processes

Earliest Eligible Virtual Deadline First (EEVDF)

Lag

- It is also a fair scheduler, based on the concept of *vruntime*
- Introduces the concept of **lag**, i.e., how much the *vruntime* of a process is behind or beyond the average vruntime of all running processes
  - *Example:* Assuming *Processes A* and *B* have a *vruntime* of 10 and 30, and are the only running processes in the system (average *vruntime* is 20)

$$lag = average_{vruntime} - process_{vruntime}$$

  - *Process A* has lag 10, and *Process B* has lag -10
- **Eligibility criteria:** only processes with positive lag can be scheduled
  - But which one should be scheduled next?

Earliest Eligible Virtual Deadline First (EEVDF)

Deadline

- **Selection criteria:** Instead of choosing the process with the highest lag, choose the process with the **earliest virtual deadline**

$$Virtualdeadline = eligibletime + \frac{timeslice}{weight}$$

- The *eligible time* is the current *vruntime* for the process
- Processes may request specific *timeslices* or use a default value
  i.e., allows tasks to receive shorter time slices if they are deemed latency-sensitive
- The *weight* is calculated from the process's *nice* value

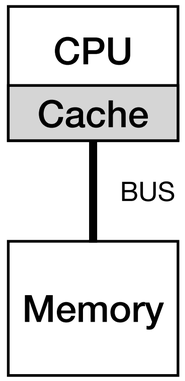
$$99.3 = 99 + \frac{1000}{3121}$$

Process	weight	vruntime	Lag	Timeslice	VD	Status
A (NICE 0)	1024	106	-6	4000 US	109.9	INELIGIBLE (LAG <0)
B (NICE -5)	**3121**	**99**	**1**	1000 US	**99.3**	**ELIGIBLE, EARLIEST VD**
C (NICE 0)	1024	98	2	2000 US	99.9	ELIGIBLE
D (NICE +5)	335	97	3	4000 US	108.9	ELIGIBLE

Multiprocessor Scheduling

Challenges

- Modern computers have multiple processors, and thus
  - How should the OS schedule processes on multiple CPUs?
  - What new problems arise?
  - Do the same old techniques work, or are new ideas required?
- As you know, a CPU has a dedicated hierarchy of **hardware caches**
  - Faster than accessing RAM!
  - Hold copies of popular memory (RAM) data from processes

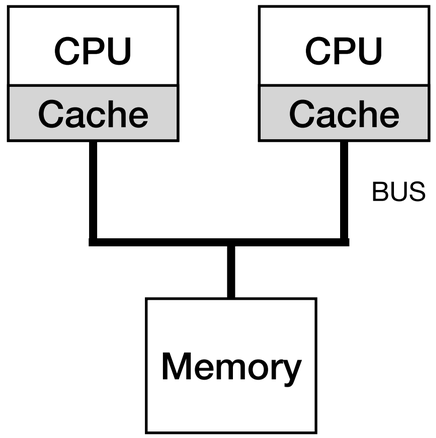


Single CPU with cache

Multiprocessor Scheduling

Cache coherence

- **Cache coherence** must be maintained when using multiple CPUs!
  - *Process A* accesses RAM's *address A*, which is fetched to CPU1's cache and later modified by the process
  - Later, Process A is rescheduled to CPU2, and accesses *address A* from main memory since it is not in CPU2's cache
  - If the modified address was not flushed from CPU1's cache to main memory, an **old (incoherent) value** is fetched!
- Therefore, the hardware must provide cache coherence protocols *i.e.*, to **invalidate** or **update** cached items at the different CPUs

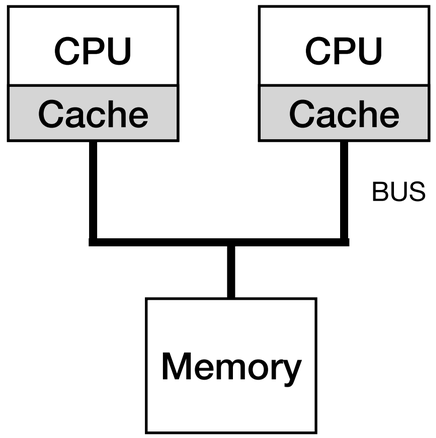


Two CPUs with caches

Multiprocessor Scheduling

Cache affinity

- When a process runs on a given CPU, it will
  - Fill the CPU caches with the process's state
  - Along with the CPU's TLB
- The next time the process runs, if the CPU caches still hold process information, it would be faster to schedule it to that specific CPU
  - Thus, the OS scheduler should consider **cache affinity** (i.e., keeping the a process in the same CPU if possible)

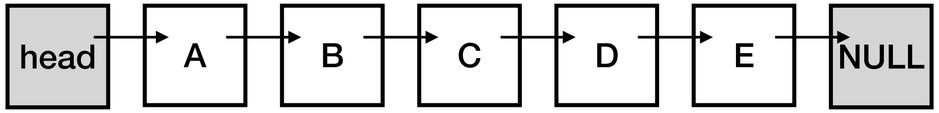


Two CPUs with caches

Multiprocessor Scheduling

Single-Queue Multiprocessor Scheduling (SQMS)

- Have a single queue from which each CPU picks the next job to run
  - **Simplicity:** can reuse existing single-CPU scheduling policies to pick the next job(s) to run
- Challenges:
  - Requires locking (concurrent accesses to the queue to pick the next element)
  - Processes may end up bouncing across different CPUs (poor affinity)
    - Solution: Include an **affinity mechanism in the scheduler**
      Complex to implement!



Queue

CPU 0	A	E	D	C	B
CPU 1	B	A	E	D	C
CPU 2	C	B	A	E	D
CPU 3	D	C	B	A	E

Execution without affinity

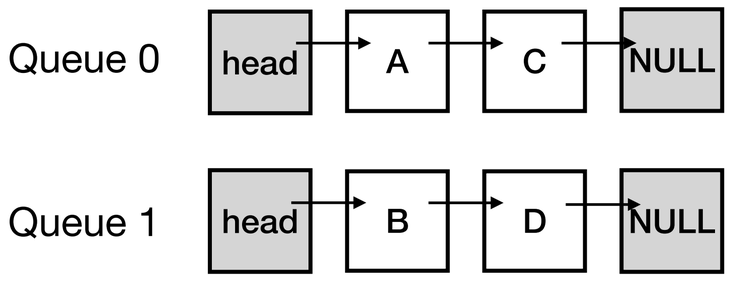
CPU 0	A	E	A	A	A
CPU 1	B	B	E	B	B
CPU 2	C	C	C	E	C
CPU 3	D	D	D	D	E

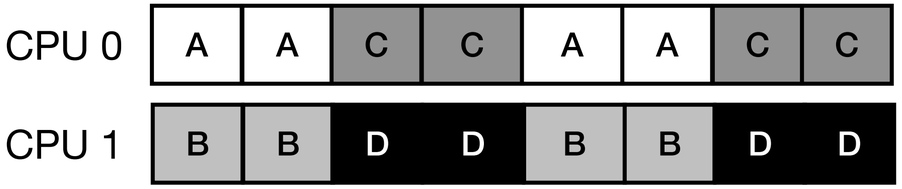
Execution with affinity
Only *Process E* migrates through CPUs

Multiprocessor Scheduling

Multi-Queue Multiprocessor Scheduling (MQMS)

- **One** scheduling **queue per CPU**
  - Each queue may follow a distinct policy
  - When a process enters the system, it is placed on a queue (*e.g.*, random picking, the queue with fewer processes)
- Avoids synchronization (global locking)
  - Multiple queues and not a single shared one
    Less contention, thus a more **scalable** design!
  - Process stays in the same CPU (queue)
    Cache **affinity**!
- What if *Process C* finishes?



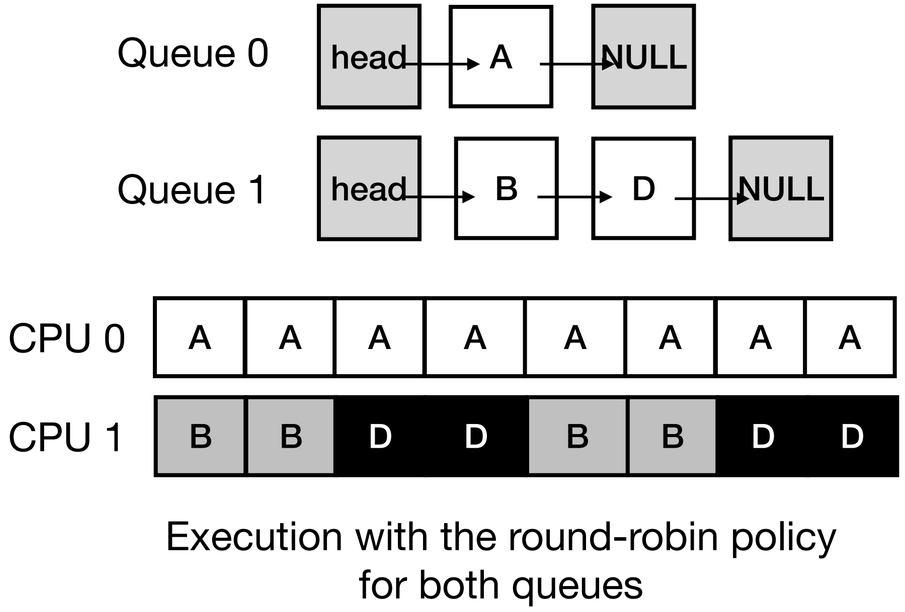


Execution with the round-robin policy for both queues

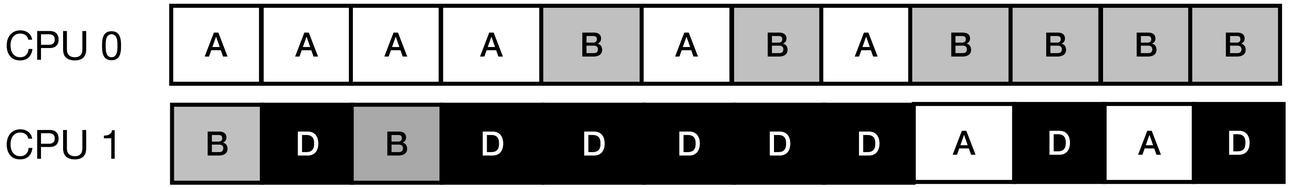
Multiprocessor Scheduling

Multi-Queue Multiprocessor Scheduling (MQMS)

- MQMS may lead to **load imbalance**
  - *Process A* gets twice the CPU time! Worst case, if Process A finishes, CPU 0 becomes idle!
- The OS scheduler must **migrate** processes across CPUs to balance the load
- Queues can periodically pick at other queues and **steal** processes if these are fuller
  - Picking requires locking, creating performance contention…
  - Looking at queues less often may lead to severe load imbalances…



Execution with the round-robin policy for both queues



Process migration example

Multiprocessor Scheduling

Linux schedulers

- There are several proposals for Linux multi-processor schedulers
- Using multiple queues
  - **O(1) scheduler:** priority-based scheduling, resembling the Multi-Level Feedback Queue (MKFQ) design
  - **CFS scheduler** (discussed previously)
  - **EEVDF scheduler** (discussed previously)
- Using a single queue
  - **BFS scheduler:** proportional-sharing scheduling based on EEVDF

More Information

- **Chapters 9 and 10** - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. **Operating Systems: Three Easy Pieces.** Arpaci-Dusseau Books, 2018.
- **An EEVDF CPU scheduler for Linux** - Jonathan Corbet. https://lwn.net/Articles/925371/, 2023
- **Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation** - Ion Stoica, Hussein Abdel-Wahab. Technical Report TR-95-22, 1996

Questions?